فصل ۱۰: شبکه‌های عصبی

شبکه‌های عصبی و یادگیری عمیق اخیراً بسیار محبوب شده‌اند زیرا مسائل سختی را برطرف کرده و شاید به بخش قابل توجهی از چهره هوش مصنوعی تبدیل شده‌اند. در این فصل شبکه‌های عصبی feed-forward موجود در scikitlearn را بررسی می‌کنیم.

کلاسه‌بند پرسپترون

با استفاده از scikit، می‌توانید کلاسه‌بندی پرسپترون را کاوش کرده و آن را با سایر روش‌های طبقه‌بندی در scikit-learn مرتبط کنید. علاوه بر این، پرسپترون عناصر سازنده شبکه‌های عصبی هستند، که بخش بسیار برجسته‌ای از یادگیری ماشین هستند.

شروع به کار

این فرایند به شرح زیر است:

- مجموعه داده‌های طبقه‌بندی دیابت UCI را بارگیری کنید.
- مجموعه داده را به مجموعه‌های آموزشی و آزمایشی تقسیم کنید.
- یک پرسپترون وارد کنید.
- یک شی گیرنده (پرسپترون) را ایجاد کنید.
- سپس پرسپترون را آموزش دهید.
- پرسپترون را روی مجموعه تست امتحان کنید یا ترجیحاً cross_val_score را محاسبه کنید

X مجموعه ویژگی‌های ورودی وy متغیری را که می‌خواهیم پیش‌بینی کنیم می‌باشند.
X و y را به مجموعه آزمایش و آموزش تقسیم کنید. این کار را با طبقه‌بندی مجموعه هدف انجام دهید، کلاس‌ها را به نسبت متعادل در هر دو مجموعه آموزش و آزمایش نگه دارید:

```
import numpy as np
import pandas as pd
data_web_address ="pima-indians-diabetes.csv"
column_names = ['pregnancy_x',
'plasma_con',
'blood_pressure',
'skin_mm',
'insulin',
'bmi',
'pedigree_func',
'age',
'target']
feature_names = column_names[:-1]
all_data = pd.read_csv(data_web_address , names=column_names)
X = all_data[feature_names]
y = all_data['target']
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2,stratify=y)
```

عملیات مقیاس‌گذاری را فقط روی مجموعه آموزش انجام دهید و سپس با مجموعه تست ادامه دهید

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
scaler.fit(X_train)
X_train_scaled = scaler.transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

حال یک نمونه پرسپترون را ایجاد کنید:

```
from sklearn.linear_model import Perceptron
pr = Perceptron()
pr.fit(X_train_scaled, y_train)
Perceptron(alpha=0.0001, class_weight=None, eta0=1.0,fit_intercept=True,
n_jobs=1, penalty=None, random_state=0, shuffle=True,verbose=0,
warm_start=False)
```

امتیاز اعتبارسنجی متقابل را اندازه بگیرید.

```
from sklearn.model_selection import cross_val_score,StratifiedKFold
```

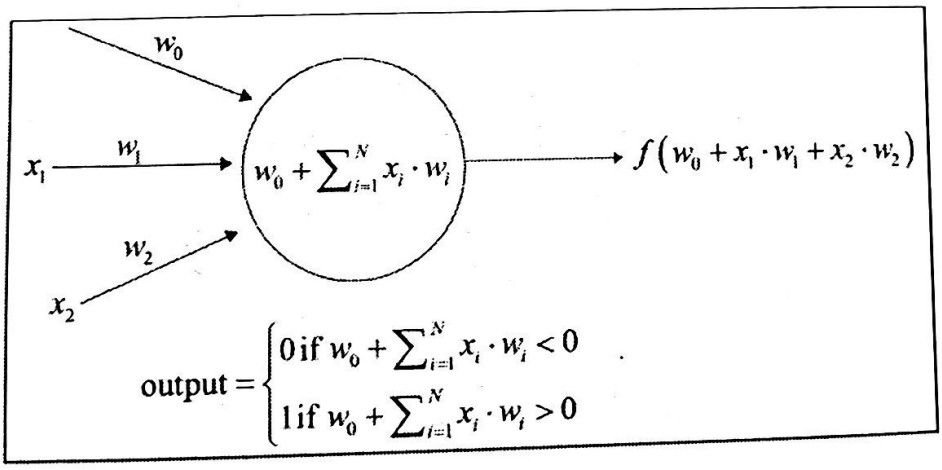
```
skf = StratifiedKFold(n_splits=3)
cross_val_score(pr, X_train_scaled, y_train,cv=skf,scoring='roc_auc').mean()
output: 0.7255833449973941
```

عملکرد را روی مجموعه تست اندازه‌گیری کنید.

```
from sklearn.metrics import accuracy_score, roc_auc_score
print ("Classification accuracy : ",
accuracy_score(y_test,pr.predict(X_test_scaled)))
print ("ROC-AUC Score : ",roc_auc_score(y_test,pr.predict(X_test_scaled)))
output:
Classification accuracy : 0.7272727272727273
ROC-AUC Score : 0.7048148148148149
```

روش کار پرسپترون

پرسپترون ساده‌سازی یک نورون در مغز است. در نمودار زیر، گیرنده ورودی‌های x_1 و x_2 را از سمت چپ را دریافت می‌کند. (بایاس، w_0 و وزن‌های w_1 و w_2 محاسبه می‌شوند.) x_i و w_i یک تابع خطی تشکیل می‌دهند که خروجی آن به یک تابع فعال‌سازی منتقل می‌شود. در تابع فعال‌سازی، اگر مجموع نقطه وزن و بردار ورودی کمتر از صفر باشد، یک خروجی صفر طبقه‌بندی می‌شود. در غیر این صورت به عنوان ۱ طبقه‌بندی می‌شود.



شکل (۱۰-۱): یک پرستپترون

این فرایند از طریق چندین تکرار انجام می‌شود و هر بار وزن‌ها تنظیم می‌شوند، عملکرد از دست دادن به حداقل می‌رسد.

محققان آزمایش‌های زیادی را انجام داده‌اند و متوجه شده‌اند که آن‌ها به خوبی کار می‌کنند. در عمل، آن‌ها در حال حاضر با سخت افزارهای موجود به خوبی کار می‌کنند. با

افزایش قدرت محاسباتی، شبکه‌های عصبی و گیرنده‌ها قادر به حل مسائل دشوارتر بوده و زمان آموزش نیز کاهش می‌یابد.

با تغییر بیش از حد پارامترهای پرسپترون، یک جستجوی شبکه را اجرا می‌کنیم. برخی از پارامترهای مهم شامل پارامترهای تنظیم، penaltyand alpha, class_weight و max_iter می‌باشند. پارامتر class_weight با دادن وزن بیشتر به کلاس‌های کم بازده، به خوبی با کلاس‌های نامتعادل کار می‌کنند. پارامتر max_iter به حداکثر تعداد عبور از پرسپترون اشاره دارد. به طور کلی، هرچه مقدار آن بالاتر باشد، بهتر است، آن را روی ۵۰ قرار می‌دهیم.

```
from sklearn.model_selection import GridSearchCV
param_dist = {'alpha': [0.1,0.01,0.001,0.0001],
'penalty': [None, 'l2','l1','elasticnet'],
'random_state': [7],
'class_weight':['balanced',None],'eta0': [0.25,0.5,0.75,1.0],
'warm_start':[True,False], 'max_iter':[50], 'tol':[1e-3]}
gs_perceptron = GridSearchCV(pr, param_dist,
scoring='roc_auc',cv=skf).fit(X_train_scaled, y_train)
gs_perceptron.best_params_
{'alpha': 0.001,
'class_weight': None,
'eta0': 0.5,
'max_iter': 50,
'penalty': 'l2',
'random_state': 7,
'tol': 0.001,
'warm_start': True}
gs_perceptron.best_score_
output: 0.8110352054428928
```

پارامترهای مختلف با استفاده از "اعتبار متقابل" نتایج را بهبود بخشیده‌اند. اکنون سعی کنید از bagging با مجموعه‌ای از پرسپترون‌ها به شرح زیر استفاده کنید. با مشاهده و انتخاب بهترین پرسپترون از جستجوی شبکه شروع می‌کنیم

```
best_perceptron = gs_perceptron.best_estimator_
from sklearn.ensemble import BaggingClassifier
from sklearn.ensemble import BaggingClassifier
param_dist = {
'max_samples': [0.5,1.0],
'max_features' : [0.5,1.0],
```

```
'oob_score' : [True, False],
'n_estimators': [100],
'n_jobs':[1],
'base_estimator__alpha': [0.001,0.002],
'base_estimator__penalty': [None, 'l2','l1','elasticnet'], }
ensemble_estimator = BaggingClassifier(base_estimator = best_perceptron)
bag_perceptrons =
GridSearchCV(ensemble_estimator,param_dist,scoring='roc_auc',cv=skf,n_jobs
=1).fit(X_train_scaled, y_train)
bag_perceptrons.best_score_
output: 0.8236958189742453
bag_perceptrons.best_params_
output:
{'base_estimator__alpha': 0.001,
'base_estimator__penalty': 'elasticnet',
'max_features': 0.5,
'max_samples': 0.5,
'n_estimators': 100,
'n_jobs': 1,
'oob_score': True}
```

نتیجه حاصل بهتر از قبل است و نشان می‌دهد استفاده از bagging بهتر از استفاده از یک پرسپترون می‌باشد.

شبکه عصبی- پرسپترون چند لایه

استفاده از شبکه عصبی در scikit- learn ساده است و به صورت زیر پیش می‌رود:

- داده‌ها را بارگیری کنید.
- مقیاس داده‌ها را با یک مقیاس کننده استاندارد انجام دهید.
- جستجوی ابرپارامتر را انجام دهید. با تغییر پارامتر آلفا شروع کنید.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import fetch_california_housing
cali_housing = fetch_california_housing()
X = cali_housing.data
y = cali_housing.target
bins = np.arange(6)
binned_y = np.digitize(y, bins)
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2,stratify=binned_y)
```

با مقیاس‌گذاری متغیرهای ورودی شروع می‌کنیم.

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
scaler.fit(X_train)
X_train_scaled = scaler.transform(X_train)
```

سپس، مقیاس‌بندی را روی مجموعه آزمون انجام می‌دهیم

```
X_test_scaled = scaler.transform(X_test)
```

یک جستجوی تصادفی انجام می‌دهیم (یا اگر ترجیح می‌دهید جستجوی شبکه‌ای را انجام دهید) تا مقدار مناسبی برای آلفا پیدا کنیم

```
from sklearn.model_selection import RandomizedSearchCV
from sklearn.neural_network import MLPRegressor
param_grid = {'alpha': [10,1,0.1,0.01],
'hidden_layer_sizes' :
[(50,50,50),(50,50,50,50,50)],
'activation': ['relu','logistic'],
'solver' : ['adam']
}
pre_gs_inst = RandomizedSearchCV(MLPRegressor(random_state=7),
param_distributions = param_grid,
cv=3,
n_iter=15,
random_state=7)
pre_gs_inst.fit(X_train_scaled, y_train)
pre_gs_inst.best_score_
output: 0.7923853935927073
```

در زمینه شبکه‌های عصبی، تک گیرنده‌ها به این شکل هستند. خروجی تابعی از ضرب داخلی نقطه‌ای وزن و ورودی است. تابع fit تابع فعال‌سازی است و به عنوان مثال می‌تواند یک منحنی سیگموئید باشد. در شبکه عصبی، فعال‌سازی hyperparameter به این عملکرد اشاره دارد. در scikit- learn، گزینه identity, logistic, tanh, and relu وجود دارد، لجستیک همان منحنی سیگموئید است.

پشته‌سازی با یک شبکه عصبی

دو روش متداول meta-learning روش bagging و boosting است. روش پشته کمتر استفاده شده است با این وجود روشی قدرتمند است زیرا می‌توان مدل‌های مختلفی را با هم

ترکیب کرد. هر سه روش یک تخمین زننده قوی‌تر را از مجموعه تخمین زننده نه چندان قوی ایجاد می‌کنند. در اینجا، ما آن را با یک شبکه عصبی مخلوط و با سایر مدل‌ها امتحان می‌کنیم.

روند پشته به شرح زیر است:

۱. مجموعه داده را به مجموعه‌های آموزشی و آزمایشی تقسیم کنید.
۲. مجموعه آموزش را به دو مجموعه تقسیم کنید.
۳. در اولین قسمت از مجموعه آموزش، به یادگیر آموزش دهید.
۴. با استفاده از یادگیرهای پایه در قسمت دوم مجموعه آموزش را پیش‌بینی کنید.
۵. بردارهای پیش‌بینی را ذخیره کنید.
۶. بردارهای پیش‌بینی ذخیره شده را به عنوان ورودی و متغیر هدف را به عنوان خروجی در نظر بگیرید.
۷. یک یادگیر سطح بالاتر را آموزش دهید (توجه داشته باشید که ما هنوز در قسمت دوم مجموعه آموزش هستیم).

پس از آن، می‌توانید نتایج فرآیند کلی را روی مجموعه آزمون مشاهده کنید (توجه داشته باشید که با مشاهده نتایج در مجموعه آزمون نمی‌توانید مدلی را انتخاب کنید).

مجموعه مسکن کالیفرنیا و کتابخانه‌هایی را که از آن‌ها استفاده کرده‌ایم وارد کنید: numpy، pandas و matplotlib. این یک مجموعه داده با اندازه متوسط است اما نسبت به سایر مجموعه داده‌های دیگر بسیار بزرگ است.

```
from __future__ import division
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import fetch_california_housing
#From within an ipython notebook
%matplotlib inline
cali_housing = fetch_california_housing()
X = cali_housing.data
y = cali_housing.target
```

برای افزایش تعادل در تقسیم مجموعه داده از نظر هدف، متغیر هدف را در بخش‌هایی (سطل) قراردهید

```
bins = np.arange(6)
binned_y = np.digitize(y, bins)
```

مجموعه داده X و y را به سه مجموعه تقسیم کنید. X_1 و X_stack به ترتیب به متغیرهای ورودی مجموعه آموزش اول و دوم اشاره دارند. y_1 و y_stack به ترتیب به متغیرهای هدف خروجی مجموعه آموزش اول و دوم مراجعه می‌کنند. مجموعه آزمون از X_test_prin و y_test_prin تشکیل شده است

```
from sklearn.model_selection import train_test_split
X_train_prin, X_test_prin, y_train_prin, y_test_prin = train_test_split(X,
y,test_size=0.2,stratify=binned_y,random_state=7)
binned_y_train_prin = np.digitize(y_train_prin, bins)
X_1, X_stack, y_1, y_stack
=train_test_split(X_train_prin,y_train_prin,test_size=0.33,stratify=binned_y_tra
in_prin,random_state=7 )
```

گزینه دیگر که می‌توانید از آن استفاده کنید StratifiedShuffleSplit است که در ماژول model_selection در scikit-learn است.

مدل پایه اول- شبکه عصبی

در جستجو در شبکه با اعتبارسنجی متقابل در اولین مجموعه آموزش، یک شبکه عصبی اضافه کنید: X_1، ورودی‌ها و y_1 مجموعه هدف. با این کار بهترین پارامترهای شبکه عصبی برای این مجموعه داده پیدا می‌شود. ما در این مثال فقط پارامتر آلفا را تغییر می‌دهیم. فراموش نکنید که ورودی‌ها را مقیاس‌بندی کنید در غیر این صورت به جواب مناسب نخواهیم رسید.

```
from sklearn.model_selection import GridSearchCV
from sklearn.neural_network import MLPRegressor
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
mlp_pipe = Pipeline(steps=[('scale', StandardScaler()),
('neural_net', MLPRegressor())])
param_grid = {'neural_net__alpha': [0.02,0.01,0.005],
'neural_net__hidden_layer_sizes' : [(50,50,50)],
'neural_net__activation': ['relu'],
'neural_net__solver' : ['adam']
}
neural_net_gs = GridSearchCV(mlp_pipe, param_grid =param_grid,cv=3,
n_jobs=1)
neural_net_gs.fit(X_1, y_1)
```

بهترین پارامتر را نمایش می‌دهیم:

```
neural_net_gs.best_params_
output: {'neural_net__activation': 'relu',
 'neural_net__alpha': 0.02,
 'neural_net__hidden_layer_sizes': (50, 50, 50),
 'neural_net__solver': 'adam'}
neural_net_gs.best_score_
output:0.7860080961517886
```

شبکه عصبی با بهترین عملکرد را ذخیره کنید، زیرا در آموزشی که انجام داده‌ایم صرفه‌جویی شود تا مجبور نشویم چندین بار آن را ادامه دهیم.

```
nn_best = neural_net_gs.best_estimator_
import pickle
f = open('nn_best.save', 'wb')
pickle.dump(nn_best, f, protocol = pickle.HIGHEST_PROTOCOL)
f.close()    .
```

مدل پایه دوم– Gradient boosting Ensemble

یک شبکه جستجوی تصادفی روی درختان gradient boost اعمال می‌کنیم

```
from sklearn.model_selection import RandomizedSearchCV
from sklearn.ensemble import GradientBoostingRegressor
param_grid = {'learning_rate': [0.1,0.05,0.03,0.01],
'loss': ['huber'],
'max_depth': [5,7,10],
'max_features': [0.4,0.6,0.8,1.0],
'min_samples_leaf': [2,3,5],
'n_estimators': [100],
'warm_start': [True], 'random_state':[7]
}
boost_gs = RandomizedSearchCV(GradientBoostingRegressor(),
param_distributions = param_grid,cv=3, n_jobs=1,n_iter=25)
boost_gs.fit(X_1, y_1)
```

بهترین امتیازها را نمایش می‌دهیم:

```
boost_gs.best_score_
output:0.8226596837997034
boost_gs.best_params_
output:{'warm_start': True, 'random_state': 7, 'n_estimators': 100,
'min_samples_leaf': 3, 'max_features': 0.4, 'max_depth': 7, 'loss': 'huber',
'learning_rate': 0.1}
```

تعداد تخمین زننده‌ها را افزایش می‌دهیم:

```
gbt_inst = GradientBoostingRegressor(**{'learning_rate': 0.1,
'loss': 'huber',
'max_depth': 10,
'max_features': 0.4,
'min_samples_leaf': 5,
'n_estimators': 4000,
'warm_start': True, 'random_state':7}).fit(X_1, y_1)
```

با کمک تابع زیر می‌توان مدل را ذخیره نمود:

```
def pickle_func(filename, saved_object):
 import pickle
 f = open(filename, 'wb')
 pickle.dump(saved_object, f, protocol =pickle.HIGHEST_PROTOCOL)
 f.close()
 return None
pickle_func('grad_boost.save', gbt_inst)
```

مدل پایه سوم – bagging regressor of gradient boost ensembles

اکنون، یک جستجوی شبکه کوچک برای gradient-boosted trees انجام می‌دهیم. از دیدگاه تئوری تعیین عملکرد آن‌ها دشوار است. به منظور پشته‌سازی، اگر با دیگر برآوردگرهای پایه همبستگی زیادی نداشته باشد، این مدل به خوبی می‌تواند جواب مناسب را بیابد.

```
from sklearn.ensemble import BaggingRegressor,GradientBoostingRegressor
from sklearn.model_selection import RandomizedSearchCV
param_dist = {
'max_samples': [0.5,1.0],
'max_features' : [0.5,1.0],
'oob_score' : [True, False],
'base_estimator__min_samples_leaf': [4,5],
'n_estimators': [20]
}
single_estimator = GradientBoostingRegressor(**{'learning_rate':
0.1,
'loss': 'huber',
'max_depth': 10,
'max_features': 0.4,
'n_estimators': 20,
'warm_start': True, 'random_state':7})
ensemble_estimator = BaggingRegressor(base_estimator =
```

```
single_estimator)
pre_gs_inst_bag = RandomizedSearchCV(ensemble_estimator,
param_distributions = param_dist,
cv=3,
n_iter = 5,
n_jobs=1)
pre_gs_inst_bag.fit(X_1, y_1)
```

حال امتیازها را مشاهده کرده و بهترین مدل را ذخیره می‌کنیم

```
pre_gs_inst_bag.best_score_
output: 0.7809440775488147
pre_gs_inst_bag.best_params_
output:{'oob_score': False,
 'n_estimators': 20,
 'max_samples': 1.0,
 'max_features': 1.0,
 'base_estimator__min_samples_leaf': 4}
pickle_func('bag_gbm.save', pre_gs_inst_bag.best_estimator_)
```

برخی از توابع پشته

تابع handle_X_set یک مجموعه داده از بردارهای پیش‌بینی در مجموعه X_stack ایجاد می‌کند. از نظر مفهومی، به مرحله چهارم پیش‌بینی قسمت دوم مجموعه آموزش اشاره دارد:

```
def handle_X_set(X_train_set_in):
  X_train_set = X_train_set_in.copy()
  y_pred_nn = neural_net.predict(X_train_set)
  y_pred_gbt = gbt.predict(X_train_set)
  y_pred_bag = bag_gbm.predict(X_train_set)
  preds_df = pd.DataFrame(columns = ['nn', 'gbt','bag'])
  preds_df['nn'] = y_pred_nn
  preds_df['gbt'] = y_pred_gbt
  preds_df['bag'] = y_pred_bag
  return preds_df
def predict_from_X_set(X_train_set_in):
  X_train_set = X_train_set_in.copy()
  return final_etr.predict(handle_X_set(X_train_set))
```

در این مرحله مدل‌های ذخیره شده را بارگذاری می‌کنیم

```
def pickle_load_func(filename):
  f = open(filename, 'rb')
```

```
  to_return = pickle.load(f)
  f.close()
  return to_return
neural_net = pickle_load_func('nn_best.save')
gbt = pickle_load_func('grad_boost.save')
bag_gbm = pickle_load_func('bag_gbm.save')
```

با استفاده از عملکرد handle_X_ یک پیش‌فرض داده ایجاد کنید. همبستگی pearson بین بردارهای پیش‌بینی را چاپ کنید

```
preds_df = handle_X_set(X_stack)
print (preds_df.corr())
output:
  nn   gbt    bag
nn  1.000000 0.958632 0.962872
gbt 0.958632 1.000000 0.981386
bag 0.962872 0.981386 1.000000
```

Meta-learner

یک مرجع درخت اضافی در دیتافریم پیش‌بینی‌ها آموزش دهید. از y_stack به عنوان بردار هدف استفاده کنید

```
from sklearn.ensemble import ExtraTreesRegressor
from sklearn.model_selection import RandomizedSearchCV
param_dist = {'max_features' : ['sqrt','log2',1.0],
'min_samples_leaf' : [1, 2, 3, 7, 11],
'n_estimators': [50, 100],
'oob_score': [True, False]}
pre_gs_inst
=RandomizedSearchCV(ExtraTreesRegressor(warm_start=True,bootstrap=True
,random_state=7),
param_distributions = param_dist,cv=3,n_iter = 15,random_state=7)
pre_gs_inst.fit(preds_df.values, y_stack)
pre_gs_inst.best_params_
output:
{'oob_score': False,
 'n_estimators': 100,
 'min_samples_leaf': 11,
 'max_features': 1.0}
```

تعداد برآوردگرها را افزایش می‌دهیم

```
final_etr = ExtraTreesRegressor(**{'max_features': 1.0,
'min_samples_leaf': 11,
```

```
'n_estimators': 3000,'oob_score': False,
'random_state':7}).fit(preds_df.values,y_stack)
```

نتایج اعتبارسنجی متقابل را مشاهده کنید

```
from sklearn.model_selection import cross_val_score
cross_val_score(final_etr, preds_df.values, y_stack, cv=3).mean()
output: 0.8248086183460068
```

عملکرد را بر روی مجموعه آزمایش مشاهده کنید

```
y_pred = predict_from_X_set(X_test_prin)
from sklearn.metrics import r2_score, mean_absolute_error
print ("R-squared",r2_score(y_test_prin, y_pred))
print ("MAE : ",mean_absolute_error(y_test_prin, y_pred))
print ("MAPE : ",(np.abs(y_test_prin- y_pred)/y_test_prin).mean())
output: R-squared 0.8409328514317185
MAE : 0.29973340499620627
MAPE : 0.16607545475815433
```

شاید بتوانیم نتایج را حتی بیشتر کنیم. ستون‌های آموزش را در کنار بردارهای پیش‌بینی قرار دهید. با اصلاح عملکردهایی که ما استفاده کرده‌ایم شروع کنید

```
def handle_X_set_sp(X_train_set_in):
  X_train_set = X_train_set_in.copy()
  y_pred_nn = neural_net.predict(X_train_set)
  y_pred_gbt = gbt.predict(X_train_set)
  y_pred_bag = bag_gbm.predict(X_train_set)
  preds_df = pd.DataFrame(X_train_set, columns =
cali_housing.feature_names)
  preds_df['nn'] = y_pred_nn
  preds_df['gbt'] = y_pred_gbt
  preds_df['bag'] = y_pred_bag
  return preds_df
def predict_from_X_set_sp(X_train_set_in):
  X_train_set = X_train_set_in.copy()
  return final_etr_sp.predict(handle_X_set_sp(X_train_set))
```

آموزش رگرسیون درختان را مانند قبل ادامه دهید

```
preds_df_sp = handle_X_set_sp(X_stack)
from sklearn.ensemble import ExtraTreesRegressor
from sklearn.model_selection import RandomizedSearchCV
param_dist = {'max_features' : ['sqrt','log2',1.0],
'min_samples_leaf' : [1, 2, 3, 7, 11],
'n_estimators': [50, 100],
'oob_score': [True, False]}
```

```
pre_gs_inst_2
=RandomizedSearchCV(ExtraTreesRegressor(warm_start=True,bootstrap=True
,random_state=7),
param_distributions = param_dist,cv=3,n_iter = 15,random_state=7)
pre_gs_inst_2.fit(preds_df_sp.values, y_stack)
```

ما مانند گذشته ادامه می‌دهیم. بهترین پارامترها را مشاهده کرده و مدلی با تخمین زننده بیشتر آموزش دهید

```
{'max_features': 'log2',
'min_samples_leaf': 2,
'n_estimators': 100,
'oob_score': False}
final_etr_sp = ExtraTreesRegressor(**{'max_features': 'log2',
'min_samples_leaf': 2,
'n_estimators': 3000,
'oob_score': False,'random_state':7}).fit(preds_df_sp.values,y_stack)
```

عملکرد اعتبارسنجی متقابل را مشاهده کنید

```
from sklearn.model_selection import cross_val_score
cross_val_score(final_etr_sp, preds_df_sp.values, y_stack,cv=3).mean()
output:
0.8322606997660486
```

ما ستون‌های ورودی اصلی را در آموزش یادگیرنده سطح بالا پشته گنجانده‌ایم. عملکرد اعتبارسنجی متقابل از ۰/۸۲۴۸ به ۰/۸۳۳۲۲ افزایش یافته است. بنابراین، نتیجه می‌گیریم که مدلی که شامل ستون‌های ورودی است بهترین مدل است. پس از انتخاب این مدل به عنوان بهترین مدل نهایی، به عملکرد برآوردگر روی مجموعه آزمایش نگاه می‌کنیم.

```
y_pred = predict_from_X_set_sp(X_test_prin)
from sklearn.metrics import r2_score, mean_absolute_error
print ("R-squared",r2_score(y_test_prin, y_pred))
print ("MAE : ",mean_absolute_error(y_test_prin, y_pred))
print ("MAPE : ",(np.abs(y_test_prin- y_pred)/y_test_prin).mean())
output:
R-squared 0.8475233336907441
MAE : 0.29254590050431367
MAPE : 0.16132256472694165
```

منابع

- Avila Julian , Hauck Trent, *scikit-learn cook book*,Packt,2017
- Hackeling Gavin, Mastering Machine Learning with Scikit-Learn,Packt, 2014
- IEEE, Arthur Lee Samuel, https://history.computer.org/pioneers/samuel.html, 2013
- Tekir Selma, Mansmann Florian, ,Mathematics, *Geodesic distances for web document clustering* ,Computer Science 2011, IEEE Symposium on Computational Intelligence and Data Mining (CIDM)
- Stephanie Glen. "P-Value in Statistical Hypothesis Tests: What is it?" From StatisticsHowTo.com: Elementary Statistics for the rest of us! https://www.statisticshowto.com/probability-and-statistics/statistics-definitions/p-value/
- Ding Shifei, Hui Li, Chunyang Su, Junzhao Yu & Fengxiang Jin, *Evolutionary artificial neural networks: a review*,Springer, 2011,
- Vinod Kumar Chauhan, Kalpana Dahiya & Anuj Sharma, *Problem formulations and solvers in linear SVM: a review*, Springer, 2019
- Pourghasemi Hamid Reza, Teimoori Yansari Zeinab, Panos Panagos & Biswajeet Pradhan, *Analysis and evaluation of landslide susceptibility: a review on articles published during 2005–2016 (periods of 2005–2012 and 2013–2016)* , Springer, 2018
- Yildiz B., Bilbao J.I., Sproul A.B., *A review and analysis of regression and machine learning models on commercial building electricity load forecasting*, Renewable and Sustainable Energy Reviews,Volume 73, Elsevier, 2017
- Ayşe Neslihan Şener, M. Erdem Günay, Aybüke Leba, Ramazan Yıldırım, *Statistical review of dry reforming of methane literature using decision tree and artificial neural network analysis*,Catalysis Today,Volume 299,2018.